

Daffodil On-Demand Diagnostics

Design and implementation notes — generic remote-triggered diagnostics over LoRa, built out from the V50_I TX-current investigation.

Why this exists

While validating V50_I (raw solar/USB input, sensed on ADS1115 AIN3) as a cloudiness signal, a discrepancy showed up: an unloaded USB voltmeter read 5.2 V on the panel while the onboard ADC read 3.37 V under real operating conditions. Tracing the schematic showed why — the MCP73123 battery charger (IC2) draws its supply directly off the same V50_I net the ADC senses, so the ADC reading reflects the panel *under load*, not open-circuit. That's a feature, not a bug: it's a better cloudiness signal than an unloaded voltmeter reading precisely because it captures real harvest, not ambient light.

The next question — whether a LoRa send draws from the battery or the panel — needed something the firmware didn't have: visibility into current *during* the actual over-the-air window, which is normally hidden inside a blocking `LoRa.endPacket(false)` call. That need for one-off, non-UART-dependent instrumentation is what grew into a general on-demand diagnostics framework, since the same need (test something specific, on demand, per device type, without a USB cable in hand) will come up again for other measurements.

Logic

1. Trigger (remote, over LoRa)

Diagnostics are turned on/off remotely using the existing `RequestCommand` packet type (already used for commands like `SendCurrentData` and `SetTroughParameters#...`) — no new packet type needed for the trigger, only for the result.

Command string	Effect
<code>EnableDiagnostics#1</code>	Sets <code>rtc_activeDiagnosticType = 1 (DIAGNOSTIC_TYPE_TX_CURRENT)</code> and <code>rtc_diagnosticsEnabled = true</code>
<code>DisableDiagnostics</code>	Clears both back to <code>DIAGNOSTIC_TYPE_NONE / false</code>

Both flags are declared `RTC_DATA_ATTR`, the same convention used elsewhere in this firmware for state that must survive deep sleep (alongside `rtc_comma_mode`, `rtc_has_comma_data`, etc.). That matters here specifically: Daffodil deep-sleeps between wake cycles, so a plain global would silently reset to "off" on the very next sleep, defeating the "enable once, stays on until explicitly disabled" behavior the remote command implies.

2. Capture (during the next — and every subsequent — LoRa send)

Inside the templated `sendMessage(T)` function, the send path already had to change to see the current-during-TX phenomenon at all:

```
LoRa.beginPacket();
LoRa.write(...);
LoRa.endPacket(true);           // async, not blocking
while (LoRa.isTransmitting()) {
    sample = ina219.getCurrent_mA(); // ~1ms per I2C read at 100kHz
    ...
}
```

Swapping the blocking `endPacket(false)` for `async endPacket(true)` + a tight `isTransmitting()` poll loop turns the previously-opaque TX window into a series of timestamped current samples — roughly a dozen-plus samples across a typical ~100–300 ms SF9 airtime. This path now serves two independent consumers of the same loop:

- **Serial debug** (gated on the existing `debug` flag) — prints each sample live, for when a UART cable is available.
- **Diagnostic capture** (gated on `rtc_diagnosticsEnabled && rtc_activeDiagnosticType == DIAGNOSTIC_TYPE_TX_CURRENT`) — writes each sample into a RAM buffer, `pendingTxDiagnostic`, capped at 20 entries.

Either, both, or neither can be active on a given send — they don't depend on each other.

3. Report (a second, dedicated LoRa send)

Immediately after the normal periodic `DigitalStablesData` send completes, if diagnostics are active and samples were captured, the firmware assembles a `DiagnosticRecord` from `pendingTxDiagnostic` and sends it as its own LoRa packet — a second, small transmission right behind the first, rather than trying to cram diagnostic data into the already near-maximum-size main packet.

Why not enlarge `DigitalStablesData` instead? It's already close to the practical LoRa payload ceiling (~248 bytes on this radio/SF configuration). A dedicated, occasional diagnostic packet avoids growing the size — and therefore the airtime and power cost — of *every* normal send just to carry data that's only relevant while someone is actively debugging.

Data structure

`DiagnosticRecord` lives in the shared `DigitalStablesData.h` header (used by every Digital Stables device firmware, not just Daffodil), following the same packed/fixed-layout style already established there by `GraveyardShiftUpdate`.

```
DiagnosticRecord                                     // 101 bytes, #pragma pack(push,1)
├─ totpcode          long          (4B)    // auth, same TOTP scheme as other packets
├─ serialnumberarray[8] uint8_t[8] (8B)    // which device this diagnostic is from
├─ diagnosticType     uint8_t       (1B)    // selects the active union member below
├─ payload            union         (87B)   // sized to its largest member
│   └─ txCurrent : TxCurrentDiagnosticPayload
│       └─ v50i_mv    uint16_t      (2B)    // V50_I just before key-up, millivolts
```

```
|      |— mAPre          uint16_t  (2B)  // battery current just before key-up
|      |— mApost        uint16_t  (2B)  // battery current just after TX ends
|      |— sampleCount   uint8_t   (1B)  // valid entries in samples[]
|      |— samples[20] : TxCurrentSample // (20 × 4B = 80B)
|      |      |— offsetMs   uint16_t (2B)  // ms since LoRa.beginPacket()
|      |      |— milliamps  uint16_t (2B)  // INA219 current_mA at that offset
|— checksum            uint8_t   (1B)  // XOR checksum, same scheme as DigitalStablesData
```

Why a union, not a growing struct

Every future diagnostic type (RSSI-over-time, I2C error latency, whatever comes up next) gets its own payload struct and a new `DIAGNOSTIC_TYPE_*` constant, added as another member of the same union — not a new top-level LoRa packet type. That keeps the receiver's packet-size dispatch (`packetSize == sizeof(DigitalStablesData)`, etc.) unchanged: there is exactly one new size to recognize, `sizeof(DiagnosticRecord)`, regardless of how many diagnostic kinds exist behind it.

Trade-off, noted at design time: because `LoRa.write()` sends `sizeof(T)` raw bytes regardless of which union member is populated, every `DiagnosticRecord` transmission costs the airtime of the *largest* diagnostic payload ever defined — not just the one actually in use. Accepted deliberately here because diagnostics are on-demand and rare, not part of the periodic uplink; the alternative (a distinct struct — and therefore a distinct packet size — per diagnostic type) was rejected as more receiver-side complexity than the airtime savings justified.

Existing LoRa packet types (for size-collision reference)

Struct	Approx. size	Purpose
GraveyardShiftUpdate	19 bytes	Overnight battery forecast, Annabelle → field device
RequestCommand	~37 bytes	Text command dispatch (this is what triggers diagnostics)
DiagnosticRecord	101 bytes	New — this document
WeatherForecastUpdate	~170+ bytes	4× forecast records
DigitalStablesData	~248 bytes	Main periodic sensor/status uplink

End-to-end flow

```
human
  → TeleonomeWebApp
    → RequestCommand ("EnableDiagnostics#1")
      → MapBusThread
        → Async Message mutation
          → Annabelle — LoRa → Daffodil
                                sets rtc_diagnosticsEnabled=true,
                                rtc_activeDiagnosticType=1

(next periodic wake)
```

```
Daffodil: sendMessage(digitalStablesData)
    └─ async TX poll captures pendingTxDiagnostic
        sendMessage(diagnosticRecord) — LoRa → Annabelle
                                           (receive-side: not yet built)

... later ...
human → ... → RequestCommand ("DisableDiagnostics") → Annabelle — LoRa → Daffodil
```

Status

Built: shared `DiagnosticRecord/TxCurrentDiagnosticPayload` struct definitions; Daffodil-side command `dispatch(EnableDiagnostics#N / DisableDiagnostics)`; async TX-current capture inside `sendMessage()`; follow-up `DiagnosticRecord` transmission after the main periodic send.

Not yet built: Annabelle's send side (issuing the `EnableDiagnostics/DisableDiagnostics` commands) and receive side (recognizing a 101-byte packet and doing something with a `DiagnosticRecord`) — plus everything upstream of Annabelle (TeleonomeWebApp UI, `MapBusThread`, the async message mutation). None of that has been inspected or touched yet.